

Procedures and the Stack

AArch64 calls, stack frames, parameter passing, and AAPCS64

Computer Organization & Architecture | AArch64 Reference

Topic 8 Outline

Topic Objective

Explain how AArch64 implements function calls using branch-with-link, return addresses, stack discipline, and register-preservation rules.

- **Section 1:** Procedure calls and returns
- **Section 2:** Argument passing and return values
- **Section 3:** Caller-saved and callee-saved registers
- **Section 4:** Stack frames, prologues, and epilogues
- **Section 5:** Leaf functions, non-leaf functions, and recursion
- **Section 6:** Summary and self-test

Procedure Calls and Returns

Why Procedures Need a Convention

Problem

A function call temporarily transfers control to another code region, but the caller must later continue as if the call produced one well-defined result.

- The caller and callee need an agreed way to pass arguments.
- The return address must be saved somewhere.
- Both sides must know which registers may be overwritten.
- The stack must remain correctly aligned and balanced.

Calling Convention

The A64 ISA supplies instructions such as `bl`, `blr`, and `ret`. AAPCS64 is the software contract that assigns argument registers, preserved registers, return values, and stack rules.

Branch with Link

Function Call Instruction

`bl function`

- `bl` branches to the target label.
- The address of the next instruction is written to `x30`.
- Register `x30` is also called the link register, `lr`.
- The callee uses that saved address to return to the caller.

Architectural Effect

`bl` changes control flow and records the return address. It does not automatically save general-purpose registers or allocate a stack frame.

Return from a Procedure

Return Instruction

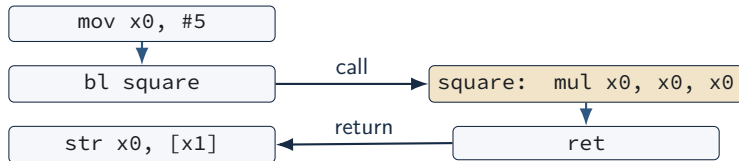
`ret`

- By default, `ret` branches to the address held in `x30`.
- It completes the control-flow path started by `bl`.
- The return value, if any, is usually already placed in `x0`.
- The stack pointer must be restored before returning.

Common Form

Most simple returns use `ret`. The explicit form `ret x30` means return to the address in `x30`.

Call and Return Control Flow



Control-Flow Rule

The call target runs temporarily, then `ret` resumes execution after the original `bl` instruction.

The Link Register Hazard

Why Non-Leaf Functions Need Care

A function that calls another function will execute its own bl. That new bl overwrites x30.

Leaf Function

- Calls no other function.
- May return using the original x30.
- May not need a stack frame.

Non-Leaf Function

- Calls at least one function.
- Must preserve its return address if needed later.
- Commonly saves x30 on the stack.

Practical Rule

If a function uses bl and still needs its caller's return address, it must save x30 before the nested call.

Arguments and Return Values

Integer Argument Registers

AArch64 Procedure Call Standard

The first eight ordinary integer or pointer arguments use x0–x7; 32-bit values use the corresponding w0–w7 views.

Register	Common role at a function call
x0–x7	Integer/pointer arguments; x0 also holds a common return value.
x8	Indirect result location in some ABI-defined cases.
x9–x15	Temporary registers available for general use.
x16, x17	Intra-procedure-call scratch registers used by veneers/linker-generated code.
x18	Platform register; portable code should not treat it as an ordinary temporary.

Arguments Beyond the Register Limit

More Than Eight Arguments

After x0–x7 are assigned, additional integer or pointer arguments are passed in memory through the stack.

- The caller creates the required argument area.
- The first stacked argument begins at the incoming sp.
- Later stacked arguments follow at ABI-defined aligned offsets.
- The caller removes its outgoing argument area after the call.

Nine-Argument Shape

```
// a1-a8 use x0-x7
sub sp, sp, #16
str x9, [sp]
bl combine9
add sp, sp, #16
```

Here x9 temporarily holds the ninth argument before the caller stores it at the incoming stack location for combine9.

Return Values

Common Integer Return

`x0 = return value`

- A simple integer or pointer return value is placed in `x0`.
- Some larger or multi-register return values may use additional ABI rules.
- The caller reads the return value after the callee executes `ret`.
- If the caller needs an argument value after the call, it must preserve it if the register is caller-saved.

Example

```
bl abs_value
str x0, [x20]
```

Example Function Call

C

```
z = add2(a, b);
```

- First argument: a
- Second argument: b
- Return value assigned to z

AArch64 Shape

```
mov x0, x19  
mov x1, x20  
bl add2  
mov x21, x0
```

The example assumes a, b, and z are being kept in x19, x20, and x21.

Simple Leaf Function

C Function

```
long add2(long a, long b)
return a + b;
```

AArch64 Function Body

```
add2:
add x0, x0, x1
ret
```

- x0 and x1 arrive holding the two arguments.
- The sum is written to x0, the return-value register.
- No stack frame is required for this simple leaf function.

Register Preservation Rules

AAPCS64 Is the Procedure-Call Blueprint

Arm Procedure Call Standard for AArch64

AAPCS64 is the engineering blueprint that lets assembly routines, compilers, libraries, linkers, and debuggers cooperate correctly.

The Blueprint Defines

- where arguments and results are placed
- which registers may change or must be preserved
- how sp and stack data are aligned

Why the Rules Matter

- independently produced code can interoperate
- callers and callees know their responsibilities
- violations can corrupt data or return control

Two layers: The A64 ISA defines what instructions can do; AAPCS64 defines how software must use those capabilities at procedure boundaries.

Caller-Saved and Callee-Saved Registers

Category	Registers	Meaning
Caller-saved	x0–x17	A call may overwrite them; the caller saves live values it still needs.
Callee-saved	x19–x28	The callee must restore these before returning if it changes them.
Platform register	x18	Its use is platform-specific; portable code should avoid it.
Frame pointer	x29	Preserved when used as the current frame pointer.
Link register	x30	Holds the return address after <code>bl</code> ; non-leaf functions commonly save it.

Core Distinction

Caller-saved values may be destroyed by a call; callee-saved values must be restored by the function that changes them.

A libc Call Uses the Same AAPCS64 Contract

C call with nine total arguments: `printf(format, v1, v2, ..., v8);`

AAPCS64 Placement

Location	Arg. or result
x0	address of format
x1-x7	values v1-v7
[sp]	v8: ninth total argument
w0	characters printed on return

Caller Setup

```
// x19 = format; x20-x27 = v1-v8
sub sp, sp, #16
str x27, [sp] // v8
mov x0, x19 // format
mov x1, x20 // v1
// repeat through x7 = v7
bl printf
add sp, sp, #16
```

Why it works: libc finds every argument because both sides follow AAPCS64. The surrounding non-leaf function preserves x30 separately.

Caller-Saved Example

Situation

The caller has a live value in x0, but also needs to call another function. Since x0 is caller-saved, the caller must protect it if the value is still needed.

```
str x0, [sp, #-16]!  
bl helper  
ldr x1, [sp], #16
```

- This example reserves 16 bytes to preserve stack alignment.
- The result from `helper` remains in x0; the saved value is restored into x1.
- The principle is that the caller saves caller-saved values it still needs.

Callee-Saved Example

Situation

A function wants to use x19. Because x19 is callee-saved, the function must restore the caller's original x19 before returning.

```
str x19, [sp, #-16]!  
// use x19 inside function  
ldr x19, [sp], #16  
ret
```

Preservation Rule

The caller should see the same x19 value after the function returns as before the call began.

Common Register Roles

Register(s)	Typical ABI role
x0–x7	Integer/pointer arguments and return values.
x9–x15	Temporary caller-saved registers.
x16, x17	Intra-procedure-call temporaries; veneers may overwrite them.
x18	Platform register with platform-specific use.
x19–x28	Callee-saved registers.
x29	Frame pointer, when a frame pointer is used.
x30	Link register, holding a return address after <code>bl</code> .
sp	Stack pointer; 16-byte aligned at public interfaces and when used for memory access.

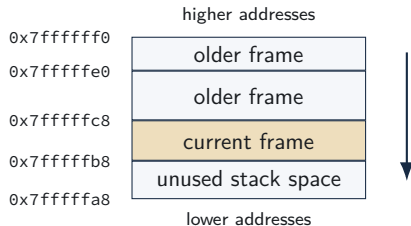
Stack Frames

The Stack Grows Down

- The stack is a memory region used for call-related temporary storage.
- Under AAPCS64, `sp` is 16-byte aligned at public interfaces and whenever memory is accessed through `sp`.
- Pushing data usually subtracts from `sp`.
- Popping data usually adds back to `sp`.

Direction

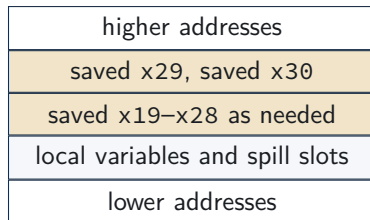
The stack grows toward lower addresses, so allocating stack space means decreasing `sp`.



illustrative only — frame sizes vary by function

Stack Frame Contents

- Saved return address when x30 must be preserved.
- Saved frame pointer when x29 is used.
- Saved callee-saved registers used by the function.
- Local variables or spill slots that do not fit in registers.
- Outgoing stack arguments when register arguments are insufficient.



sp after allocation

Function Prologue

Purpose

A prologue prepares the callee's execution environment before the function body runs.

```
    stp x29, x30, [sp, #-16]!  
    mov x29, sp
```

- The first instruction allocates 16 bytes and stores old x29 and x30.
- `mov x29, sp` establishes a frame pointer for this frame.
- Additional stack allocation may follow if locals or spills are needed.

Compiler Variation

Depending on the platform ABI profile and optimization choices, code may omit x29 as a frame pointer or use a different frame layout while preserving its obligations.

Function Epilogue

Purpose

An epilogue restores the caller's environment before returning.

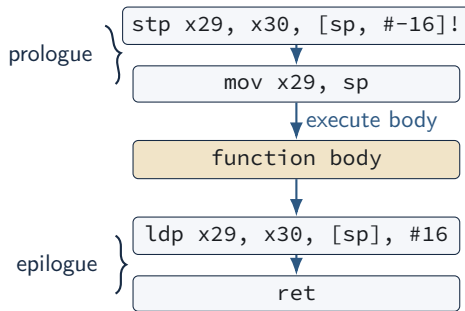
```
ldp x29, x30, [sp], #16  
ret
```

- The load pair restores the previous frame pointer and return address.
- Post-indexing adds 16 to sp, releasing the stack space.
- `ret` branches to the restored return address in x30.

Stack Balance

A function that subtracts from sp must add the same amount back before returning.

Prologue and Epilogue Diagram



Function Entry and Exit

The prologue establishes the function's stack-frame state before the body executes. The epilogue reverses those changes, restores the saved return state, and then `ret` transfers control back to the caller.

Local Stack Storage

When Registers Are Not Enough

A function may allocate additional stack space for local arrays, temporary spill slots, or values whose address must be taken.

```
sub sp, sp, #32
str x0, [sp, #16]
ldr x1, [sp, #16]
add sp, sp, #32
```

- Stack space is usually allocated in multiples that preserve alignment.
- Positive offsets from `sp` or `x29` can address slots in the frame.
- The frame layout is a compiler convention, not an A64 instruction requirement.

Function Calls and Recursion

Leaf Function without a Stack Frame

Leaf Function

A leaf function calls no other function. If it only uses caller-saved registers and needs no local stack storage, it may avoid stack use entirely.

```
double_it:  
    add x0, x0, x0  
    ret
```

- Argument arrives in x0.
- Return value is written back into x0.
- x30 is not overwritten because this function makes no nested calls.

Non-Leaf Function Saves the Link Register

Nested Call

A non-leaf function must preserve its own return address before executing another `bl` if it needs to return to its caller afterward.

```
outer:  
    stp x29, x30, [sp, #-16]!  
    mov x29, sp  
    bl inner  
    add x0, x0, #1  
    ldp x29, x30, [sp], #16  
    ret
```

Reason

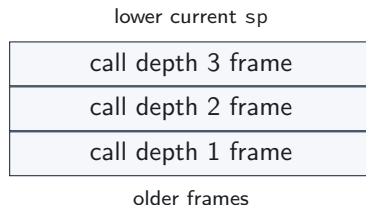
The `bl inner` instruction overwrites `x30`; therefore `outer` saves its original `x30` first.

Recursion Requires Separate Frames

Recursive Calls

Each recursive call needs its own preserved state: return address, saved registers, and local values that must survive the nested call.

- One invocation cannot reuse another invocation's saved return address.
- Local variables from different recursion depths must remain distinct.
- Stack frames naturally provide separate storage per invocation.



Recursive Factorial Uses One Frame per Call

Entry and Base Case

```
fact:
stp x29, x30, [sp, #-32]!
mov x29, sp
str x0, [sp, #16]
cmp x0, #1
b.gt recurse
mov x0, #1
b done
```

Recursive Case and Return

```
recurse:
sub x0, x0, #1
bl fact
ldr x1, [sp, #16]
mul x0, x1, x0
done:
ldp x29, x30, [sp], #32
ret
```

Each invocation preserves its return address and original n ; the recursive result returns in $x0$.

Common Procedure Bugs

- Forgetting that `b1` overwrites `x30`.
- Changing `x19–x28` without restoring them before returning.
- Returning with `sp` different from its value at function entry.
- Assuming caller-saved registers survive a call.
- Using `w` registers for pointer values that require 64-bit addresses.
- Adjusting `sp` by only 8 bytes and then accessing memory through the misaligned stack pointer.

Debugging Order

Check the return address, then stack balance, then register-preservation rules.

Summary and Self-Test

Topic 8 Summary

- `bl` calls a function and writes the return address into `x30`.
- `ret` returns to the address held in `x30` by default.
- `x0–x7` pass the first eight integer/pointer arguments; 32-bit values use `w0–w7`.
- Additional arguments use the stack, and `x0` holds a common integer/pointer return value.
- Caller-saved registers may be overwritten by a call; callee-saved registers must be restored if changed.
- Stack frames preserve return addresses, saved registers, and local storage.
- Leaf functions may avoid stack frames; non-leaf and recursive functions often need them.

Self-Test 1/2

- ❶ What does `bl` function write into `x30`?
- ❷ Why can a non-leaf function not blindly trust `x30` after it calls another function?
- ❸ Which registers carry the first eight integer or pointer arguments, and where is a ninth argument passed?
- ❹ Where is a simple integer return value normally placed?
- ❺ What is the difference between caller-saved and callee-saved registers?

Self-Test 2/2

- 6 Why does `stp x29, x30, [sp, #-16]!` both allocate stack space and save registers?
- 7 What does `ldp x29, x30, [sp], #16` do before the final `ret`?
- 8 Why must `sp` be restored before returning to the caller?
- 9 Why can a small leaf function sometimes avoid a stack frame?
- 10 Why does recursion require a separate stack frame for each active call?

Register Preservation Exercise

Classify Each Register

For each register, state whether it is mainly used for argument/return, caller-saved temporary use, callee-saved preservation, frame pointer, link register, or stack pointer.

Register	Classification
x0	
x9	
x19	
x29	
x30	
sp	

Machine Instruction Encoding

Topic 9 moves from assembly-level procedure structure to the binary encoding of selected A64 instructions.

- AArch64 instruction formats and machine-code fields
- Data-processing instruction encoding
- Memory instruction encoding and PC-relative addressing
- Branch instruction encoding and machine-code decoding